



*Campus  
Mexicali*

**“Las ciencias  
Básicas son el  
cimiento de la  
ingeniería”**

**07-Mayo-2026**

## **INSTITUTO TECNOLÓGICO DE MEXICALI**

### **Carrera:**

ING. SISTEMAS COMPUTACIONALES.

### **Materia:**

PROGRAMACIÓN ORIENTADA A OBJETOS

### **Tema:**

EXCEPCIONES

### **Alumno:**

José Manuel Monge Delmoral

### **Profesor(a):**

Marisela Ponce Millanes.

## ¿Qué es una excepción?

Una **excepción** es un evento anómalo que interrumpe el flujo normal de ejecución de las instrucciones de un programa. Técnicamente, es un objeto que encapsula información sobre un error que ocurre debido a un estado inválido, como el manejo de referencias nulas (`NullPointerException`) o violaciones de restricciones aritméticas, como la **división por cero**. En Java, estos eventos se gestionan mediante una estructura de control específica denominada bloque `try-catch`.

## Importancia del manejo de excepciones

La gestión adecuada de excepciones es fundamental para garantizar la **robustez** y la **tolerancia a fallos** de una aplicación. Permite definir un "flujo alternativo" o plan de contingencia que puede:

1. **Recuperar el estado:** Corregir el error en tiempo de ejecución para retomar el flujo habitual.
  2. **Degradación controlada:** Si el error no es recuperable, permite cerrar recursos de forma segura.
  3. **Feedback informativo:** Proporcionar al usuario final o al desarrollador mensajes precisos (logs) sobre la causa raíz del fallo, facilitando la resolución de errores recurrentes.
- 

## Clasificación de Excepciones

### 1. Excepciones Verificadas (Checked Exceptions)

Son aquellas que el compilador obliga a gestionar. Derivan de la clase `Exception` (pero no de `RuntimeException`). Se detectan en **tiempo de compilación** porque representan escenarios que el programa debe prever, como la ausencia de un archivo externo (`FileNotFoundException`) o problemas de red. Si no se capturan o se declaran, el código no compilará.

### 2. Excepciones No Verificadas (Unchecked Exceptions)

Ocurren en **tiempo de ejecución** (Runtime) y derivan de la clase `RuntimeException`. Generalmente son causadas por errores en la lógica del programador o datos de entrada inválidos que no fueron validados previamente. El compilador no obliga a escribirlas en un `try-catch`, pero de no manejarse, provocarán la finalización abrupta del programa.

---

## Estructura Try-Catch-Finally

Es el mecanismo estándar para el control de excepciones:

- **Try:** Bloque que contiene el código "sensible" que podría disparar una anomalía.
  - **Catch:** Bloque encargado de interceptar el objeto de la excepción para procesarlo. Se pueden tener múltiples bloques catch para diferentes tipos de error.
  - **Finally:** Bloque opcional que se ejecuta **siempre**, independientemente de si ocurrió una excepción o no. Es ideal para la liberación de recursos (cerrar bases de datos o archivos).
- 

## Escenarios Comunes de Excepciones

- **Aritmética inválida:** Intentar una división por cero (`ArithmeticException`).
- **Uso de métodos matemáticos:** Intentar calcular la raíz cuadrada de un número negativo con tipos de datos enteros (aunque en `Math.sqrt` devuelve NaN, en otros lenguajes o contextos lanza error).
- **Violación de límites:** Acceder a un índice fuera del rango de un arreglo (`ArrayIndexOutOfBoundsException`).
- **Incompatibilidad de tipos:** Insertar un objeto de tipo erróneo en una colección o pasar un dato no numérico a un `Scanner` que espera un entero (`InputMismatchException`).
- **Referencia nula:** Intentar acceder a un método de un objeto que aún no ha sido instanciado.

## Ejemplo de sintaxis que causaría una excepción:

```
if (i = 2) {Cuerpo...}
```

Esta es una excepción verificada habitual conocida como “**Type mismatch**”, donde el tipo de dato que estamos pasando dentro del **if (int)** no puede ser correctamente leído ya que se espera un tipo de dato **boolean**. Es un error habitual de escritura al omitir un segundo “=” cuando se quiere escribir el operador lógico de comparación “==”.